Pendientes de Javier

Frontend y revisión independiente del backend

Punto de partida

La aplicación ya está conectada a la API y Atlas. Las evidencias de índices y de integración están integradas en main mediante el PR #4. No hace falta rehacer el frontend: falta ejecutar una revisión propia y dejar sus resultados.

1. Actualizar y preparar el entorno

Desde una copia sin cambios pendientes, actualizar main con git pull --ff-only. Usar Node 24. Entrar a backend y ejecutar npm ci. Registrar el commit revisado con git rev-parse --short HEAD.

2. Ejecutar las pruebas aisladas

Dentro de backend, ejecutar uno por uno:

```
npm test
npm run evidence:integration
npm run evidence:indexes
```

Resultado esperado: 10 pruebas en npm test; 48 solicitudes HTTP y 4 comprobaciones MongoDB en integración. En índices: tres consultas conservan resultados/orden y examinan menos documentos; el caso sin categoría mantiene SORT.

Estas pruebas usan MongoDB temporal local: no requieren .env ni credenciales de Atlas. El primer uso puede descargar MongoDB 7.0.24. Los comandos de evidencia regeneran JSON y documentos; revisar el diff antes de subir archivos.

3. Revisar la interfaz con la base compartida

Coordinar con Matías. Configurar backend/.env por privado, iniciar backend con npm start y frontend con npm ci y npm run dev. Abrir http://localhost:5173.

Crear un cliente y producto ficticios identificables; crear un pedido; comprobar total del servidor, historial al editar el catálogo, estados y reporte. Probar email duplicado, bajas lógicas y persistencia al recargar.

Evidencias y cierre

Qué entregar para completar la revisión

4. Comprobar comportamiento y accesibilidad

Revisar mensajes de carga, listas vacías y errores de red/HTTP. Confirmar que un error no cierre el formulario ni borre lo escrito. Probar búsqueda, filtros y paginación. Revisar móvil, etiquetas de campos, foco y uso con teclado.

Comprobar el comportamiento sin backend y que Reintentar carga recupere los datos al volver a iniciarlo. Al finalizar, dar de baja solo los registros ficticios propios, sin borrar documentos ajenos.

5. Dejar registro independiente

Completar docs/revision-cruzada.md cuando esté disponible en GitHub, o entregar una ficha equivalente. Incluir fecha, sistema operativo, versión de Node, commit revisado, salida de comandos y flujos de interfaz realmente ejecutados.

Anotar cada problema con pasos para reproducir, resultado esperado y resultado obtenido. Terminar con un veredicto: aprobado, aprobado con observaciones o requiere cambios. La ejecución anterior del autor no reemplaza esta revisión.

6. Capturas para el informe

Entregar capturas legibles de listado y formulario, pedido con detalle histórico, error de validación y reporte. Incluir resultados de pruebas o logs sanitizados. No mostrar contraseñas, URI con credenciales ni .env.

Criterio para dar tu parte por cerrada

Los tres comandos se ejecutaron en tu equipo, los flujos quedaron verificados, las observaciones están registradas y la aprobación es tuya. El grupo puede usar esa evidencia en el informe y la defensa.

Preguntas para ensayar

¿Por qué el navegador no usa la URI de Atlas? ¿Por qué el total definitivo lo calcula el servidor? ¿Por qué cambiar un producto no cambia el pedido anterior? ¿Qué distingue COLLSCAN, IXSCAN y SORT?

Archivos de referencia

docs/pruebas-integracion.md; docs/evidencias/integracion/http.json; docs/evidencias/indices/README.md; frontend/README.md; docs/contrato-api.md.

Estado de este documento: guía de tareas; no certifica que la revisión de Javier ya se haya realizado.